

How to attach Prometheus and Micrometer telemetry to a multi-module KMP Project



Standard black-box telemetry fails: Standard APMs and network interceptors treat the client/backend boundary as an opaque box. Did you know you can integrate custom telemetrics that exposes the lifecycle of long-lived connections and coroutine suspensions?

Setup: Multimodule Ktor project w/ Prometheus + Micrometer?

- Captures more than just Websocket handshakes: also captures every inbound/output command
- Coroutine context preserved across suspensions
- Allows for custom tag setup

Sample implementation example:

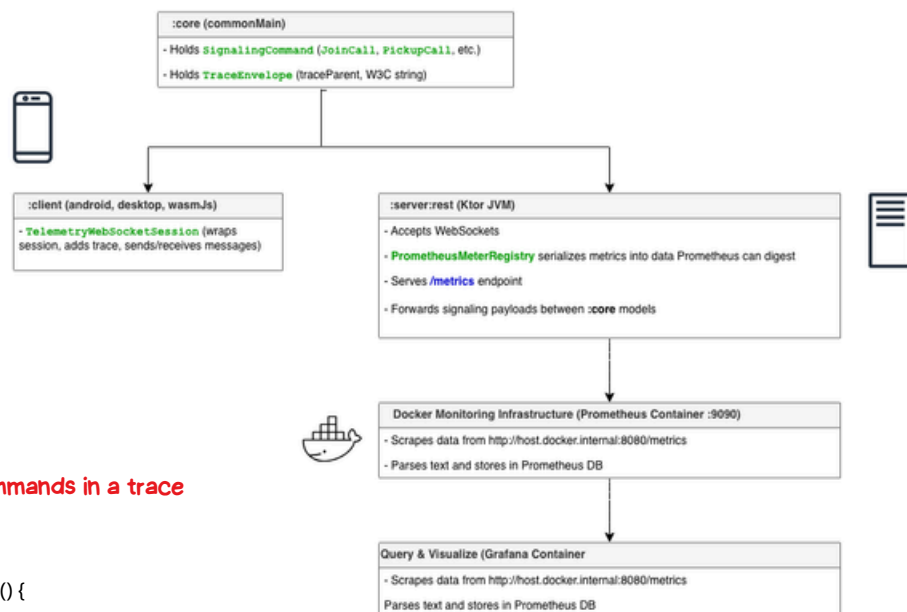
github.com/ahinchman1/ktor-chat/pull/1

Dependencies needed:

- Micrometer: github.com/micrometer-metrics/micrometer
- Prometheus: github.com/prometheus/prometheus

Implementation plan

- **[client]** wrap telemetry around ktor websocket to attach traces to payloads
- **[server:rest]** install micrometer plugin into Prometheus registry
- **[docker-compose.yml]:** Set up Compose container for Prometheus data scraping
- **[3000]** configure Prometheus data source pointing for Grafana visualizations



1. client - extend DefaultWebSocketSession to wrap commands in a trace

```
class TelemetryWebSocketSession(  
    private val delegate: DefaultClientWebSocketSession  
) : DefaultWebSocketSession by delegate, SignalingTelemetry() {  
  
    suspend fun sendCommand(command: SignalingCommand) {  
        val tracedCommand = traceCommand(command)  
        delegate.sendSerialized(tracedCommand)  
    }  
  
    suspend fun receiveCommand(): SignalingCommand {  
        return delegate.receiveDeserialized()  
    }  
}
```

2. client - intercept outgoing/incoming signals with extended special class

```
private suspend fun connectOnce(scope: CoroutineScope, token: String) {  
    val block: suspend DefaultClientWebSocketSession.() -> Unit = {  
        val telemetrySession = TelemetryWebSocketSession(this)  
        telemetrySession.onConnect(scope)  
    }  
  
    if (enableSecureConnection) {  
        http().wss(path = "call", request = { withAuth(token) }, block = block)  
    } else {  
        http().ws(path = "call", request = { withAuth(token) }, block = block)  
    }  
}
```

```
suspend fun TelemetryWebSocketSession.onConnect(  
    scope: CoroutineScope  
) {  
    val sendJob = scope.launch {  
        _outgoingCommands.collect { sendCommand(it) }  
    }  
    println("Listening for commands on WebSocket")  
  
    runCatching {  
        while (true) {  
            val command = receiveCommand()  
            println("Received command: $command")  
            _signalingCommandFlow.emit(command)  
        }  
    }.onFailure { it.printStackTrace() }  
    sendJob.takeIf { it.isActive }?.cancel()  
}
```

How to attach Prometheus and Micrometer telemetry to a multi-module KMP Project

server - install micrometer plugin into Prometheus registry

```
val appMicrometerRegistry = PrometheusMeterRegistry(PrometheusConfig.DEFAULT)
```

```
fun Application.rest() {  
    install(CORS) {  
        install(WebSockets) {  
            contentConverter = KotlinWebsocketSerializationConverter(signalingCommandsFormat)  
        }  
    }  
}
```

```
install(MicrometerMetrics) {  
    registry = appMicrometerRegistry  
    meterBinders = listOf(  
        JvmMemoryMetrics(),  
        JvmGcMetrics(),  
        ProcessorMetrics()  
    )  
}  
  
routing {  
    get("/metrics") {  
        call.respond(appMicrometerRegistry.scrape())  
    }  
}
```

```
val activeConnections = appMicrometerRegistry.gauge(  
    "chat.connections.active",  
    AtomicInteger(0)  
)  
val messageCounter = appMicrometerRegistry.counter("chat.messages.received")
```

docker-compose.yml setup

version: '3.8'

services:

...

prometheus:

image: prom/prometheus:latest

restart: unless-stopped

ports:

- "9090:9090"

volumes:

- ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro

extra_hosts:

- "host.docker.internal:host-gateway"

grafana:

image: grafana/grafana:latest

restart: unless-stopped

ports:

- "3000:3000"

environment:

- GF_SECURITY_ADMIN_USER=admin

- GF_SECURITY_ADMIN_PASSWORD=admin

- GF_USERS_ALLOW_SIGN_UP=false

volumes:

- ./provisioning/datasources:/etc/grafana/provisioning/datasources:ro

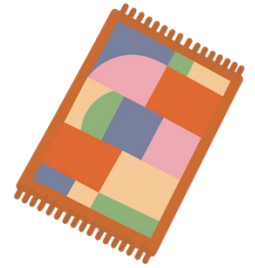
- ./provisioning/dashboards/dashboards.yml:/etc/grafana/provisioning/dashboards/dashboards.yml:ro

- ./provisioning/dashboards:/var/lib/grafana/dashboards:ro

depends_on:

- prometheus

add additional yaml files for persistence, dashboards, data sources, and more



Instructions to run everything together to get Grafana live during runtime

1. Start background services

`docker compose up -d`

2. Start local Ktor server

`./gradlew :server:rest:run`

3. Run the Desktop App

`./gradlew :app:desktop:jvmRun`

4. Run the Android app

Android Studio w/ emulator works well

5. Navigate to

`http://localhost:8080/metrics` for metrics data

6. Navigate to `http://localhost:3000` for Grafana

